TREASURY CONNECT

AI-Powered Contact Center Platform

Kiro CLI Implementation Specification

Amazon Connect + Amazon Nova Sonic + Amazon Bedrock + AWS AgentCore

FedRAMP Moderate | U.S. Department of the Treasury

Document Information	
Version	1.0
Date	March 2026
Classification	AWS Confidential
Author	Chad — Principal SA, Global AI/GenAI CX Lead
Target Bureaus	IRS, TreasuryDirect, TOP, U.S. Mint, Direct Express
Deployment	us-east-1 / us-west-2 (FedRAMP Moderate)
Kiro CLI Version	1.x (Latest)

Table of Contents

1. Project Overview and Kiro Configuration
2. System Requirements (requirements.md)
3. AWS Service Architecture Map
4. Design Specification (design.md)
5. CDK Infrastructure Stacks
6. Implementation Tasks (tasks.md)
7. Amazon Connect Contact Flow Design
8. Amazon Nova Sonic Speech Pipeline
9. Amazon Bedrock AI Agent Configuration
10. Knowledge Base and RAG Pipeline
11. Three-Layer Governance System
12. Deterministic Prediction Engine
13. Skills-Based Routing Engine
14. Post-Escalation Agent Lifecycle
15. Real-Time Analytics Pipeline
16. Flow Operations Center Dashboard
17. Authentication and Identity
18. Security, Compliance, and FedRAMP
19. Testing and Validation
20. Appendix: Complete File Manifest

1. Project Overview and Kiro Configuration

Treasury Connect is an AI-powered contact center platform built on Amazon Connect, serving five U.S. Treasury bureaus (IRS, TreasuryDirect, Treasury Offset Program, U.S. Mint, Direct Express) with real-time speech-to-speech AI self-service, three-layer deterministic governance, skills-based routing, and a comprehensive operations dashboard.

This document serves as the complete Kiro CLI specification for building the production system. Kiro uses three core spec files — requirements.md, design.md, and tasks.md — to drive AI-assisted development. Each section maps to Kiro's spec-driven workflow.

1.1 Kiro Project Initialization

```
kiro init treasury-connect
cd treasury-connect
kiro config set language typescript
kiro config set framework cdk
kiro config set region us-east-1
```

1.2 Repository Structure

```
treasury-connect/
  .kiro/
    requirements.md
    design.md
    tasks.md
  infrastructure/
    lib/
      stacks/
        connect-stack.ts          # Amazon Connect instance + flows
        nova-sonic-stack.ts        # Speech-to-speech pipeline
        bedrock-stack.ts           # AI agents + knowledge bases
        agentcore-stack.ts         # AgentCore + Cedar policies
        governance-stack.ts        # AR + Guardrails
        data-stack.ts              # DynamoDB + OpenSearch + S3
        analytics-stack.ts         # Kinesis + Data Lake + QuickSight
        auth-stack.ts              # Cognito + Login.gov SAML
        network-stack.ts           # VPC + PrivateLink + WAF
        monitoring-stack.ts        # CloudWatch + alarms
        dashboard-stack.ts         # Flow Ops Center (CloudFront + S3)
    bin/
      treasury-connect.ts          # CDK app entry point
  lambdas/
```

```
    auth/                          # Identity verification functions
    tools/                         # Bureau-specific tool functions
    prediction/                    # Deterministic prediction engine
    routing/                       # Skills-based routing engine
    analytics/                     # Real-time analytics processors
    governance/                    # AR validation + policy evaluation
  connect-flows/                   # Contact flow JSON exports
  knowledge-base/                  # KB documents per bureau
  cedar-policies/                  # AgentCore Cedar policy files
  ar-policies/                     # Automated Reasoning policy domains
  guardrails/                      # Bedrock Guardrails configurations
  dashboard/                       # Flow Ops Center React app
  tests/
    unit/
    integration/
    e2e/
```

1.3 Key Performance Targets

Metric	Target	Measurement	Source
Self-Service Containment	70-85%	Contacts resolved without agent	Connect Analytics
Speed to Answer	<30 seconds	Time from ingress to AI response	CloudWatch
First Contact Resolution	90%+	Issues resolved in single contact	Cases + Analytics
CSAT Score	85%+	Post-contact survey results	Contact Lens + Survey
Cost Reduction	40-60%	vs traditional IVR + agent model	Cost Explorer
AI Response Latency	<2 seconds	End-to-end tool + RAG + governance	X-Ray traces
AR Verification Rate	100%	All factual responses AR-verified	AR audit logs
Governance Pass Rate	99%+	Responses passing all 3 layers	Guardrails metrics

2. System Requirements (requirements.md)

The following requirements map directly to the .kiro/requirements.md file. Each requirement has a unique ID, acceptance criteria, and maps to specific AWS services.

2.1 Functional Requirements

FR-001: Multi-Bureau Contact Ingress

The system SHALL accept voice, chat, and SMS contacts for five Treasury bureaus (IRS, Bureau of Fiscal Service/TreasuryDirect, Treasury Offset Program, U.S. Mint, Direct Express) through Amazon Connect with bureau-specific contact flows, phone numbers, and chat widgets.

> **NOTE:** Each bureau has its own DNIS/DID, chat widget configuration, and flow entry point. Bureau identification happens in the INIT phase via SetAttributes.

FR-002: Speech-to-Speech AI Self-Service

Voice contacts SHALL be processed through Amazon Nova Sonic for real-time speech-to-speech AI interaction with barge-in support, configurable end-of-speech silence detection (1200ms default), and multi-voice support (Matthew-Neural, Lupe-Neural, Joanna-Neural, Ruth-Neural via Amazon Polly Neural TTS).

FR-003: Conversational Identity Verification

The system SHALL support four authentication methods: (1) Conversational KBA — natural language SSN last 4, DOB, ZIP verification, (2) DTMF KBA — keypad entry for phone, (3) Login.gov SAML — federated SSO for highest assurance, (4) ANI Match — phone number matching for lowest friction. Method selection is channel-dependent and risk-aware.

> **NOTE:** Amazon Connect Voice ID is deprecated and must NOT be used. Authentication is conversational KBA + Login.gov SAML + ANI matching + Contact Lens behavioral analytics.

FR-004: Intent Detection and Classification

The AI Agent SHALL classify citizen utterances into bureau-specific intents with confidence scoring. Minimum 25 intent types across all bureaus: IRS (RefundStatus, NoticeExplanation, PaymentPlan, PenaltyAbatement, FilingDeadline, W2Request, AmendedReturn), TreasuryDirect (BondValue, BondPurchase, AccountAccess, MaturityDate), TOP (OffsetInquiry, DisputeInitiation, HardshipExemption, PaymentTrace), U.S. Mint (OrderStatus, ProductAvailability, BullionPricing), Direct Express (CardBalance, TransactionDispute, ReplacementCard).

FR-005: Tool Execution with Policy Authorization

Every tool call (Lambda function invocation) SHALL be pre-authorized by AgentCore Policy (Cedar) before execution. Tool calls include bureau-specific backend lookups (IRS Master File, TreasuryDirect bond system, TOP offset database, etc.). Failed authorization SHALL block execution and log the violation.

FR-006: RAG + Rerank Knowledge Pipeline

The system SHALL use Bedrock Knowledge Bases (OpenSearch Serverless vector store) with Titan Embed Text v2 for embedding and Cohere Rerank v3.5 for result re-scoring. Minimum 12 knowledge base documents across bureaus. Contextual Grounding SHALL verify response faithfulness to retrieved content (threshold: 0.70).

FR-007: Three-Layer Governance

Every AI-generated response SHALL pass through three independent safety layers before delivery to the citizen: (1) AgentCore Policy — Cedar-based tool and action authorization, (2) Automated Reasoning — mathematical verification of facts, calculations, dates, and eligibility criteria, (3) Bedrock Guardrails — content safety, PII redaction, topic filtering, and hallucination detection. Failure in ANY layer SHALL block the response.

FR-008: Deterministic AI Prediction Engine

The system SHALL compute a self-service probability score using 10 weighted features: intentComplexity (0.22), sentimentRisk (0.16), authFriction (0.08), channelComplexity (0.05), toolHealth (0.08), kbRelevance (0.08), arConfidence (0.06), repeatCaller (0.05), queuePressure (0.12), agentAvailability (0.10). Thresholds: >62% SELF_SERVICE, 38-62% AT_RISK, <38% ESCALATE.

FR-009: Skills-Based Routing

Non-contained contacts SHALL be routed using skills-based matching. The system SHALL maintain skill catalogs per bureau (IRS 7 skills, TreasuryDirect 4, TOP 4, U.S. Mint 3, Direct Express 3), proficiency levels P1-P4, real-time queue metrics, callback evaluation (offer when wait >2 min or queue depth >8), priority boost for VERY_NEGATIVE sentiment or high complexity, and orphaned intent detection.

FR-010: Post-Escalation Agent Lifecycle

Escalated contacts SHALL progress through five agent states: TRANSFERRING, CONNECTED, HANDLING, WRAPPING, COMPLETE. Agents SHALL receive screen pop with full AI transcript, KB recommendations, AR results, and case history. Contact Lens SHALL monitor agent calls in real-time. Agent Assist SHALL push recommendations during handling.

FR-011: Flow Operations Center Dashboard

A real-time operations dashboard SHALL display: live contact list with phase/agent state indicators, 8-tile metrics bar, CloudWatch log viewer, sequence diagram generator, AI Agent/RAG/Governance/Routing/Analytics tabs, annotated interaction transcripts with 12 entity highlight classes, post-call analytics with CSAT prediction, and a guided step-by-step demo walkthrough mode.

FR-012: Real-Time and Post-Call Analytics

The system SHALL generate live call analytics (sentiment timeline, issues detected, actions timeline, AI Agent Assist recommendations) and post-call analytics (CSAT prediction, outcome summary, compliance notes, AI performance scores) for every contact.

2.2 Non-Functional Requirements

NFR-001: FedRAMP Moderate Compliance

All AWS services used SHALL be FedRAMP Moderate authorized in commercial regions (us-east-1, us-west-2). Data at rest encrypted with AWS KMS (FIPS 140-2 validated). Data in transit encrypted with TLS 1.2+. All API calls logged to CloudTrail. IAM policies follow least-privilege principle.

NFR-002: Performance

End-to-end AI response latency SHALL not exceed 2 seconds (p95) including tool execution, RAG retrieval, reranking, and governance checks. Nova Sonic speech-to-speech SHALL maintain <500ms first-byte latency. The system SHALL handle 1,000 concurrent contacts per bureau.

NFR-003: Availability

The system SHALL maintain 99.99% uptime for the contact flow path (Connect + Nova Sonic + Lambda). Multi-AZ deployment required. Automated failover for all stateful components.

NFR-004: Audit and Compliance

Every AI decision, tool call, policy evaluation, AR verification, and governance check SHALL be logged to CloudWatch with structured metadata. Complete audit trail SHALL satisfy IRS Section 6103, Privacy Act, IG, and GAO requirements. Logs retained for 7 years.

3. AWS Service Architecture Map

Every AWS service needed for the production deployment, organized by functional domain.

3.1 Contact Center Core

Service	Purpose	Configuration
Amazon Connect	Contact center platform — inbound voice, chat, SMS routing, agent desktop, contact flows	Instance per region, 5 bureau queues, skills-based routing profiles, agent hierarchy, hours of operation, security profiles
Amazon Connect Contact Lens	Real-time and post-call speech analytics — sentiment, talk-over, empathy detection, PII redaction	Enabled per instance, real-time rules for sentiment alerts, post-call analytics for all contacts, categories per bureau
Amazon Connect Cases	Case management for escalated contacts — create, track, resolve	Case templates per bureau, custom fields (bureau, intent, AR result, prediction), SLA timers
Amazon Connect Customer Profiles	Unified citizen profile — ANI matching, contact history, preferences	Profile domain, identity resolution, phone number matching, attribute mapping to bureaus
Amazon Q in Connect	Agent assist — real-time knowledge recommendations pushed to agent desktop	Knowledge base integration, intent-based recommendation triggers, confidence thresholds

3.2 AI and Machine Learning

Service	Purpose	Configuration
Amazon Nova Sonic	Speech-to-speech real-time AI — understanding + generation in single model, barge-in, streaming	Model: amazon.nova-sonic-v1, endSilenceTimeout: 1200ms, bargeIn: true, voices: 4 Polly Neural
Amazon Bedrock Agents	AI agent orchestration — tool use, multi-turn conversation, session management	Agent per bureau, action groups mapped to Lambda tools, session TTL 30min, guardrail association
Amazon Bedrock Knowledge Bases	RAG — semantic search over Treasury policy documents	Vector store: OpenSearch Serverless, embedding: Titan Embed Text v2, chunking: 512 tokens with 20% overlap
Cohere Rerank v3.5	Result re-scoring — rerank KB retrieval results by query relevance	Model: cohere.rerank-v3-5, top_n: 4, return_documents: true
Amazon Bedrock Guardrails	Content safety — PII redaction, topic filters, content filters, contextual grounding	Guardrail per bureau, PII: SSN/DOB/account redact, topics: deny political/investment, grounding threshold: 0.70
AWS AgentCore	Policy-based agent governance — Cedar authorization for tool calls and actions	Policy store with 6 Treasury Cedar policies, real-time evaluation, audit logging
Automated Reasoning	Mathematical verification — prove correctness of facts, calculations, dates, eligibility	6 AR policy domains (IRS tax guidance, notice explanations, payment plans, bond rules, offset rules, identity

		verification), 47 rules per domain
Amazon Titan Embed Text v2	Text embedding model for Knowledge Base vector search	Dimension: 1024, normalize: true

3.3 Compute and Storage

Service	Purpose	Configuration
AWS Lambda	Serverless compute — auth verification, tool execution, prediction engine, routing logic, analytics processing	Runtime: Python 3.12, memory: 512-1024MB, timeout: 30s, VPC-attached for data access, layers for shared libs
Amazon DynamoDB	NoSQL — contact state, session data, agent assignments, routing decisions, prediction cache	On-demand capacity, TTL for sessions (30 min), GSI on bureau+timestamp, point-in-time recovery, encryption at rest
Amazon OpenSearch Serverless	Vector store for Bedrock Knowledge Bases — semantic search index	Collection type: VECTORSEARCH, encryption: KMS, network: VPC endpoint, OCU: auto-scaling 2-10
Amazon S3	Object storage — KB documents, call recordings, analytics exports, CloudFormation templates, dashboard assets	Bucket per function (kb-docs, recordings, analytics, dashboard), versioning, lifecycle policies, SSE-KMS
Amazon ElastiCache (Redis)	Caching — session state, tool response cache, queue metrics snapshot	Cluster mode, r6g.large, Multi-AZ, encryption in-transit + at-rest, 1 hour TTL for tool cache

3.4 Integration and Messaging

Service	Purpose	Configuration
Amazon EventBridge	Event bus — contact lifecycle events, agent state changes, governance decisions trigger downstream	Custom bus: treasury-connect-events, rules for contact.created, contact.resolved, agent.connected, governance.failed
Amazon Kinesis Data Streams	Real-time streaming — Contact Lens output, contact events, analytics data	Shard count: auto-scaling, retention: 24h, enhanced fan-out for analytics consumers
Amazon Kinesis Data Firehose	Delivery — stream contact events to S3 data lake and OpenSearch for dashboards	Delivery stream per data type, S3 destination with Parquet conversion, buffer: 60s/5MB
Amazon SNS	Notifications — critical alerts (governance failures, AR INVALID, sentiment critical), agent notifications	Topics per severity level, SMS/email subscriptions for ops team
Amazon SQS	Queue — async processing for post-call analytics, case creation, follow-up scheduling	Standard queues, DLQ for failures, visibility timeout: 5 min
AWS Step Functions	Orchestration — multi-step governance pipeline, callback scheduling, case creation workflow	Express workflows for governance (< 5 min), Standard for case lifecycle

3.5 Security and Identity

Service	Purpose	Configuration
Amazon Cognito	User pools — dashboard authentication, API authorization, Login.gov SAML federation	User pool with SAML IdP (Login.gov), custom attributes (bureau, role, clearance), MFA required
AWS IAM	Access control — least-privilege roles for Lambda, Connect, Bedrock, all services	Service-linked roles, policy conditions on bureau tags, session policies for agent access
AWS KMS	Key management — encryption keys for all data at rest, FIPS 140-2 Level 2 validated	CMK per data classification (PII, operational, analytics), automatic rotation, key policy per service
AWS Secrets Manager	Secret storage — API keys, database credentials, integration tokens	Automatic rotation every 30 days, cross-account access denied, VPC endpoint access only
AWS WAF	Web firewall — protect dashboard and API endpoints from attacks	Web ACL: rate limiting (2000 req/5min), geo-restriction (US only), SQL injection + XSS rules, bot control
AWS Certificate Manager	TLS certificates — HTTPS for all endpoints	Public certificates for CloudFront, private CA for internal services

3.6 Networking

Service	Purpose	Configuration
Amazon VPC	Network isolation — Lambda, OpenSearch, ElastiCache in private subnets	3 AZ, /16 CIDR, public/private/isolated subnets, flow logs to CloudWatch
AWS PrivateLink	Private connectivity — VPC endpoints for Bedrock, S3, DynamoDB, KMS, CloudWatch	Interface endpoints for Bedrock, Gateway endpoints for S3/DynamoDB, DNS resolution enabled
Amazon CloudFront	CDN — serve Flow Ops Center dashboard, API acceleration	OAC for S3, custom domain, HTTPS-only, cache policy for static assets, WAF association
Amazon Route 53	DNS — custom domains for dashboard and API	Hosted zone, alias records for CloudFront, health checks, failover routing

3.7 Monitoring and Observability

Service	Purpose	Configuration
Amazon CloudWatch	Metrics, logs, alarms — all service monitoring, custom metrics for prediction/governance	Log groups per service, metric filters for error rates, dashboards for ops, 7-year retention
AWS X-Ray	Distributed tracing — end-to-end latency for contact flow path	Sampling: 5%, trace Lambda + Bedrock + DynamoDB, service map visualization
Amazon CloudWatch Logs Insights	Log analytics — query across all log groups for incident	Saved queries for common patterns (AR failures, policy denials, escalation

	investigation	analysis)
AWS CloudTrail	API audit — all AWS API calls logged for compliance	Multi-region trail, S3 delivery, CloudWatch Logs integration, log file validation
Amazon Connect Analytics Data Lake	Contact center analytics — historical data for reporting and ML	Enabled per instance, S3 export, Athena query, QuickSight dashboards

4. Design Specification (design.md)

This section maps to .kiro/design.md and defines the architectural decisions, data flows, and component interactions.

4.1 Contact Lifecycle — 9-Phase AI Pipeline

Every contact progresses through exactly 9 phases. The phase model is deterministic — each phase has defined inputs, outputs, success/failure criteria, and governance checkpoints.

Phase	Name	AWS Services	Key Operations
INIT	Flow Initialization	Connect, Lambda	Bureau identification, session creation, channel detection, Nova Sonic activation (voice)
AUTH	Identity Verification	Lambda, DynamoDB, Cognito	KBA/DTMF/Login.gov SAML/ANI match, identity resolution via Customer Profiles
INTENT	Intent Detection	Nova Sonic, Bedrock Agent	Utterance processing, intent classification with confidence, slot extraction
TOOL	Tool Execution	AgentCore, Lambda, DynamoDB	Cedar policy check → Lambda invocation → response. Auto-retry on timeout, circuit breaker on 503
RAG	RAG + Rerank	KB, OpenSearch, Cohere Rerank	Semantic query → vector search → chunk retrieval → Cohere rerank → contextual grounding check
GOVERN	Governance Check	Guardrails, AR, AgentCore	Three-layer pipeline: (1) AgentCore Policy (2) Automated Reasoning verification (3) Bedrock Guardrails
PREDICT	AI Prediction	Lambda, Contact Lens	10-feature weighted model, sentiment from Contact Lens, self-service probability computation
ROUTE	Routing Intelligence	Connect, Lambda	Queue metrics snapshot, skill matching, proficiency routing, callback evaluation, priority boost
RESOLVE	Resolution	Connect, Cases, Analytics	Self-service completion OR agent transfer with warm handoff OR callback scheduling

4.2 Agent State Machine

Post-escalation, contacts follow a 5-state agent lifecycle managed by EventBridge state transitions:

State	Trigger	AWS Actions	Monitoring
TRANSFERRING	RESOLVE → escalate	Connect TransferToQueue, screen pop via Q in Connect	Queue wait time metric, transfer success/fail
CONNECTED	Agent accepts	Q in Connect activates Agent Assist, Contact Lens starts	Agent greeting detection, context acknowledgment
HANDLING	Agent investigating	Agent Assist pushes KB	Empathy detection, talk-

		recs, Lambda tool calls on agent behalf	over count, speaking pace
WRAPPING	Call ends	Cases create/update, disposition saved, ACW timer starts	ACW duration, case notes completeness
COMPLETE	ACW finished	Analytics finalized, CSAT survey triggered, Data Lake export	Handle time, FCR, CSAT score

4.3 Data Flow Architecture

Real-time data flows through three parallel paths:

Path 1: Contact Flow Path (synchronous) — Connect → Nova Sonic → Bedrock Agent → Lambda → AgentCore → AR → Guardrails → Connect. Latency budget: <2 seconds total.

Path 2: Analytics Path (asynchronous) — Contact Lens → Kinesis Data Stream → Lambda (enrichment) → Kinesis Firehose → S3 Data Lake + OpenSearch (dashboards).

Path 3: Event Path (event-driven) — Contact lifecycle events → EventBridge → SNS (alerts) + SQS (processing) + Step Functions (workflows) + CloudWatch (logging).

4.4 Governance Pipeline (Step Functions Express Workflow)

The governance pipeline runs as a Step Functions Express Workflow with three parallel branches that must ALL pass:

```
Start → Parallel[
  Branch 1: AgentCore Policy Check (Cedar) → ALLOWED/DENIED
  Branch 2: Automated Reasoning Verify → VALID/INVALID/NO_DATA
  Branch 3: Bedrock Guardrails Scan → PASS/FAIL (PII + Topics + Content + Grounding)
] → ALL PASS? → Deliver Response
     ANY FAIL? → Block + Regenerate + Re-verify (max 2 retries)
```

4.5 Prediction Model Schema

The prediction Lambda computes the escalation risk score from 10 measurable features. Each feature maps to specific AWS service outputs:

Feature	Weight	Range	Source	Computation
intentComplexity	0.22	0.0 - 1.0	Bedrock Agent	Intent-specific complexity from INTENTS config (e.g., RefundStatus=0.15, HardshipExemption=0.92)
sentimentRisk	0.16	0.0 - 0.88	Contact Lens	Map: POSITIVE=0.08, NEUTRAL=0.2, NEGATIVE=0.55, VERY_NEGATIVE=0.88
authFriction	0.08	0.05 - 0.25	Lambda (auth)	ANI_MATCH=0.05, LOGIN_GOV=0.10, KBA_CONV=0.15,

				KBA_DTMF=0.25
channelComplexity	0.05	0.1 - 0.22	Connect	VOICE=0.22, CHAT=0.15, SMS=0.10
toolHealth	0.08	0.05 - 0.6	Lambda (tool)	>2000ms latency=0.6, >1000ms=0.25, otherwise=0.05
kbRelevance	0.08	0.05 - 0.55	Cohere Rerank	Top rerank score <0.75=0.55, <0.85=0.20, otherwise=0.05
arConfidence	0.06	0.02 - 0.7	Automated Reasoning	VALID=0.02, INVALID=0.70, NO_DATA=0.45
repeatCaller	0.05	0.08 - 0.4	Customer Profiles	Prior contact in 7 days = 0.40, else 0.08
queuePressure	0.12	0.05 - 0.7	Connect metrics	Queue depth >10=0.70, >5=0.35, otherwise=0.05
agentAvailability	0.10	0.05 - 0.6	Connect metrics	Available agents <3=0.60, <6=0.25, otherwise=0.05

Escalation risk = sum(feature_value * weight). Self-service probability = 1 - escalation_risk. Decision: >0.62 → SELF_SERVICE, 0.38-0.62 → AT_RISK, <0.38 → ESCALATE.

5. CDK Infrastructure Stacks

Each CDK stack is independently deployable with cross-stack references via exports. Deploy order enforced by dependencies.

5.1 Stack Deployment Order

#	Stack	Dependencies	Key Resources
1	NetworkStack	None	VPC, subnets (3 AZ), NAT gateways, VPC endpoints (Bedrock, S3, DynamoDB, KMS, Logs)
2	DataStack	NetworkStack	DynamoDB tables (contacts, sessions, predictions, routing), OpenSearch collection, S3 buckets, ElastiCache cluster
3	AuthStack	NetworkStack, DataStack	Cognito user pool, Login.gov SAML IdP, Lambda authorizer, identity resolution config
4	ConnectStack	NetworkStack, DataStack, AuthStack	Connect instance, phone numbers, queues (5 bureaus), routing profiles, security profiles, hours of operation, Contact Lens, Customer Profiles, Cases, Q in Connect
5	BedrockStack	DataStack	Bedrock agents (5 bureaus), knowledge bases, Titan embedding config, Cohere rerank model access, guardrails (5 bureaus)
6	AgentCoreStack	BedrockStack	AgentCore policy store, 6 Cedar policies, action groups, tool authorization config
7	GovernanceStack	BedrockStack, AgentCoreStack	AR policy domains (6), Step Functions express workflow (governance pipeline), retry logic
8	NovaSonicStack	ConnectStack, BedrockStack	Nova Sonic model access, Connect flow integration, voice configs, barge-in settings
9	AnalyticsStack	ConnectStack, DataStack	Kinesis streams/firehose, Connect Analytics Data Lake, CloudWatch dashboards, metric filters
10	MonitoringStack	All stacks	CloudWatch alarms, X-Ray config, CloudTrail trail, SNS topics, composite alarms
11	DashboardStack	AnalyticsStack	CloudFront distribution, S3 origin (React app), WAF web ACL, Route 53 records, API Gateway for real-time data

5.2 ConnectStack Key Resources

The Connect stack is the most complex. Key resource configurations:

```
// Queues — one per bureau with skills-based routing

IRS_Queue:        skills=[IRS_TaxReturns, IRS_Notices, IRS_Payments,
IRS_Penalties, IRS_Filing, IRS_W2, IRS_Amended]

TreasuryDirect_Q: skills=[TD_Bonds, TD_Accounts, TD_Maturity, TD_Purchase]

TOP_Queue:        skills=[TOP_Offsets, TOP_Disputes, TOP_Hardship, TOP_Traces]

USMint_Queue:     skills=[MINT_Orders, MINT_Products, MINT_Bullion]

DirectExpress_Q:  skills=[DE_Balance, DE_Disputes, DE_Cards]


// Proficiency Levels

P1: Junior      — basic inquiries, status checks

P2: Standard    — moderate complexity, standard procedures

P3: Senior      — complex cases, penalty abatement, disputes

P4: SME         — hardship exemptions, audit responses, escalated disputes
```

5.3 BedrockStack — Agent Configuration

```
// Agent per bureau — each with distinct tool sets and guardrails

Agent: treasury-irs-agent-v4

  Model: Claude Sonnet 4.5 (anthropic.claude-sonnet-4-5-v1)

  KB: kb-treasury-irs-docs-v3

  ActionGroups:

    - IRS_AccountTools: [getRefundStatus, getNoticeDetails,
calculatePaymentPlan,

                          checkPenaltyEligibility, getFilingDeadlines,
requestW2,

                          getAmendedReturnStatus]

  Guardrail: treasury-irs-guardrail-v3

  Session: TTL=1800s, history=20 turns
```

5.4 AgentCore Cedar Policies

Six Cedar policies enforce deterministic authorization boundaries:

Policy	Rule	Effect
taxpayer-isolation	session.taxpayerId !== target.taxpayerId	DENY — prevents cross-taxpayer data access
bureau-boundaries	session.bureau !== tool.requiredBureau	DENY — IRS agent cannot access TreasuryDirect tools
refund-threshold	action == 'processRefund' && amount > 10000	DENY — refunds over $10K require supervisor approval
self-service-readonly	session.type == 'self-service'	DENY — AI self-service cannot

	&& action.type == 'write'	modify records
pii-logging-prevention	action == 'log' && data.contains('SSN')	DENY — prevents PII in log output
api-allowlist	tool.endpoint NOT IN approvedEndpoints	DENY — only pre-approved Lambda functions callable

6. Implementation Tasks (tasks.md)

Tasks organized by sprint/phase. Each task maps to specific files, AWS services, and acceptance criteria. Tasks are ordered by dependency.

Phase 1: Foundation (Weeks 1-3)

#	Task	Files	Acceptance Criteria
1.1	Initialize CDK project with TypeScript, configure environments (dev/staging/prod), set up cross-stack references pattern	infrastructure/bin/treasury-connect.ts, infrastructure/lib/config.ts	cdk synth succeeds, environments configurable via context
1.2	Deploy NetworkStack — VPC (3 AZ), subnets, NAT, VPC endpoints for Bedrock/S3/DynamoDB/KMS/Logs	infrastructure/lib/stacks/network-stack.ts	VPC created, endpoints reachable from private subnets, flow logs active
1.3	Deploy DataStack — DynamoDB tables (contacts, sessions, predictions, routing), S3 buckets, OpenSearch Serverless collection for KB vector store	infrastructure/lib/stacks/data-stack.ts	Tables created with GSIs, S3 buckets with encryption, OpenSearch collection ACTIVE
1.4	Deploy AuthStack — Cognito user pool, Login.gov SAML IdP configuration, Lambda authorizer for API	infrastructure/lib/stacks/auth-stack.ts, lambdas/auth/	SAML federation works, JWT tokens issued, authorizer validates correctly
1.5	Create shared Lambda layer with common utilities (logging, DynamoDB client, error handling, CloudWatch metric emission)	lambdas/layers/shared/	Layer deployed, importable by all Lambda functions

Phase 2: Connect + Voice (Weeks 4-6)

#	Task	Files	Acceptance Criteria
2.1	Deploy ConnectStack — instance, 5 phone numbers, 5 queues with skills, routing profiles, security profiles, hours of operation	infrastructure/lib/stacks/connect-stack.ts	Instance active, queues created, skills assigned, inbound calls route correctly
2.2	Create bureau-specific contact flows — INIT phase with bureau detection, SetAttributes, channel identification	connect-flows/irs-main.json, connect-flows/td-main.json, ...	Flows imported, bureau correctly identified, attributes set
2.3	Integrate Nova Sonic speech-to-speech — streaming audio, barge-in, end-of-speech detection, multi-voice Polly TTS	infrastructure/lib/stacks/nova-sonic-stack.ts, lambdas/voice/	Voice calls answered with AI, barge-in works, voice selection per bureau
2.4	Build identity verification	lambdas/auth/verify-identity.py	All 4 auth methods work,

	Lambda — KBA conversational, KBA DTMF, ANI match logic, Login.gov SAML redirect		Customer Profiles ANI lookup, SAML flow completes
2.5	Enable Contact Lens — real-time analytics, sentiment detection, PII redaction, custom categories per bureau	infrastructure/lib/stacks/connect-stack.ts (Contact Lens config)	Sentiment scores emitting, PII redacted in transcripts, categories firing
2.6	Enable Customer Profiles — citizen profile domain, ANI identity resolution, contact history linkage	infrastructure/lib/stacks/connect-stack.ts (Profiles config)	Profiles created per citizen, ANI matching resolves identity, history linked

Phase 3: AI + Knowledge (Weeks 7-10)

#	Task	Files	Acceptance Criteria
3.1	Deploy BedrockStack — 5 Bedrock agents with Claude Sonnet 4.5, action groups mapped to bureau tools, session management	infrastructure/lib/stacks/bedrock-stack.ts	Agents respond to test prompts, tool calls execute, sessions persist across turns
3.2	Build Knowledge Base — upload 12+ policy documents, configure Titan Embed v2, OpenSearch vector index, chunking (512 tokens, 20% overlap)	knowledge-base/*.pdf, infrastructure/lib/stacks/bedrock-stack.ts	KB sync succeeds, semantic queries return relevant chunks, embedding dimensions correct
3.3	Configure Cohere Rerank v3.5 — integrate with KB retrieval, top_n=4, relevance scoring, contextual grounding check (threshold 0.70)	lambdas/rag/rerank-pipeline.py	Rerank scores computed, grounding score calculated, low-relevance results filtered
3.4	Build 25+ bureau-specific tool Lambdas — IRS (7), TreasuryDirect (4), TOP (4), U.S. Mint (3), Direct Express (3), each with DynamoDB backend	lambdas/tools/irs/*.py, lambdas/tools/td/*.py, ...	Each tool returns realistic data, error handling works, latency <2s p95
3.5	Integrate Bedrock Agent with Connect — contact flow invokes agent, agent manages multi-turn conversation, tool results returned to flow	connect-flows/ai-agent-integration.json	End-to-end: voice → intent → tool → response → citizen hears answer

Phase 4: Governance (Weeks 11-13)

#	Task	Files	Acceptance Criteria
4.1	Deploy AgentCoreStack — policy store, 6 Cedar policies, tool authorization enforcement	infrastructure/lib/stacks/agentcore-stack.ts, cedar-policies/*.cedar	Cedar policies evaluate correctly, ALLOWED/DENIED logged, bureau boundaries enforced
4.2	Configure Automated Reasoning — 6 policy domains, verification rules for	ar-policies/*.json, infrastructure/lib/stacks/governance-stack.ts	AR returns VALID/INVALID with proof, INVALID blocks response, correction

	tax guidance, bond calculations, offset rules, payment plans		suggested
4.3	Configure Bedrock Guardrails — PII redaction (SSN, DOB, account #), topic denial (political, investment), content filters, contextual grounding	guardrails/*.json, infrastructure/lib/stacks/bedrock-stack.ts	PII replaced with [REDACTED], denied topics blocked, grounding below threshold blocked
4.4	Build governance Step Functions workflow — parallel 3-branch execution, ALL-pass gate, retry on failure (max 2), audit logging	infrastructure/lib/stacks/governance-stack.ts, lambdas/governance/	All 3 layers execute in parallel, any failure blocks response, retries work, complete audit trail
4.5	End-to-end governance test — verify INVALID AR triggers correction+re-verify, DENIED policy blocks tool call, guardrail catches PII leak	tests/integration/governance.test.ts	All negative test cases pass, audit logs complete, no unauthorized data leaks

Phase 5: Prediction + Routing (Weeks 14-16)

#	Task	Files	Acceptance Criteria
5.1	Build prediction engine Lambda — 10-feature computation, weighted scoring, threshold-based decision, feature logging	lambdas/prediction/compute-prediction.py	Correct scores computed, SELF_SERVICE/AT_RISK/ESCALATE decisions match expected, all features logged
5.2	Build routing engine Lambda — skill matching (25 intent→skill mappings), proficiency evaluation, callback logic, priority boost, orphan detection	lambdas/routing/route-contact.py	Skills matched correctly, proficiency minimum enforced, callbacks offered when appropriate
5.3	Integrate with Connect queues — TransferToQueue with priority, skill-based routing profile, callback scheduling via Connect outbound	connect-flows/routing-integration.json	Contacts routed to correct skill queue, priority contacts served first, callbacks made on time
5.4	Build queue metrics Lambda — real-time queue depth, wait times, agent availability by proficiency, service level, abandonment rate	lambdas/routing/queue-metrics.py	Metrics accurate vs Connect real-time API, refreshed every 15 seconds

Phase 6: Analytics + Dashboard (Weeks 17-20)

#	Task	Files	Acceptance Criteria
6.1	Deploy AnalyticsStack — Kinesis streams for contact events, Firehose to S3 data lake (Parquet), CloudWatch metric filters for KPIs	infrastructure/lib/stacks/analytics-stack.ts	Events streaming, Parquet files in S3, metrics visible in CloudWatch
6.2	Build real-time analytics Lambda — sentiment timeline,	lambdas/analytics/realtime-analytics.py	Analytics generated for every contact, CSAT scores

	issue detection, action timeline, CSAT prediction computation		computed, issues categorized by bureau
6.3	Build Flow Operations Center dashboard — React app with WebSocket for real-time data, 9-tab layout, sequence diagram, guided demo mode	dashboard/src/	Dashboard loads, real-time contact feed, all tabs render correctly, guided demo walks through all 14 phases
6.4	Deploy DashboardStack — S3 origin, CloudFront with OAC, WAF web ACL, API Gateway WebSocket for real-time feed, Cognito auth	infrastructure/lib/stacks/ dashboard-stack.ts	Dashboard accessible via HTTPS, auth required, real-time data flowing, WAF blocking attacks
6.5	Build post-call analytics — summary generation, compliance notes, AI performance scores, follow-up scheduling, Data Lake export	lambdas/analytics/post-call.py	Post-call analytics generated within 60s of contact end, stored in Data Lake, accessible via dashboard
6.6	Enable Connect Analytics Data Lake — historical reporting, Athena queries, QuickSight dashboards for management reporting	infrastructure/lib/stacks/ analytics-stack.ts	Data Lake active, Athena queries work, QuickSight dashboard with containment/CSAT/handle time trends

Phase 7: Agent Experience (Weeks 21-22)

#	Task	Files	Acceptance Criteria
7.1	Configure Q in Connect — knowledge base integration, agent assist recommendations, screen pop with AI transcript context	infrastructure/lib/stacks/ connect-stack.ts (Q config)	Agent sees recommendations, screen pop shows AI context, KB articles surfaced correctly
7.2	Build agent lifecycle tracking — EventBridge rules for 5 state transitions, CloudWatch logging, metrics emission per state	lambdas/agent/lifecycle-tracker.py	All 5 states logged, metrics emitted, state transitions visible in dashboard
7.3	Configure Cases — templates per bureau, custom fields, SLA timers, auto-creation on escalation	infrastructure/lib/stacks/ connect-stack.ts (Cases config)	Cases created on escalation, fields populated from AI context, SLA timers running

Phase 8: Security + Compliance (Weeks 23-24)

#	Task	Files	Acceptance Criteria
8.1	Deploy MonitoringStack — CloudWatch alarms (AR failure rate, governance pass rate, latency, error rate), X-Ray tracing, CloudTrail	infrastructure/lib/stacks/ monitoring-stack.ts	Alarms firing on thresholds, X-Ray traces visible, CloudTrail logging all API calls
8.2	FedRAMP Moderate validation — verify all services in authorized list, KMS encryption everywhere, VPC endpoints, WAF rules, audit logging	docs/fedramp-validation.md	All 33 FedRAMP capabilities validated, compliance matrix complete, no gaps

8.3	Penetration testing — SQL injection, XSS, prompt injection against AI agent, privilege escalation, data exfiltration attempts	tests/security/	All attacks blocked, guardrails catch prompt injection, WAF blocks web attacks, no PII leaks
8.4	Load testing — 1,000 concurrent contacts per bureau, sustained for 1 hour, measure latency/error rate/throughput under load	tests/load/	P95 latency <2s, error rate <0.1%, no service degradation, auto-scaling works

7. Amazon Connect Contact Flow Design

Each bureau has a main contact flow that orchestrates the 9-phase pipeline. Flows are exported as JSON and version-controlled.

7.1 Flow Module Structure

Modular contact flow design — each phase is a reusable module invoked from the bureau main flow:

Module	Function	Key Blocks
Treasury_Init_Module	Bureau detection, session setup, Nova Sonic init	SetAttributes, InvokeLambda (session-create), SetVoice
Treasury_Auth_Module	Identity verification orchestration	GetCustomerInput, InvokeLambda (verify-identity), CheckAttribute (auth result)
Treasury_AI_Module	Bedrock Agent invocation with multi-turn	InvokeAgent (bureau-specific), StoreResponse, CheckConfidence
Treasury_Govern_Module	Three-layer governance pipeline	InvokeLambda (governance-pipeline), CheckAttribute (pass/fail), BranchOnResult
Treasury_Route_Module	Skills-based routing decision	InvokeLambda (route-contact), TransferToQueue (with priority), ScheduleCallback
Treasury_Resolve_Module	Resolution and wrap-up	PlayPrompt/SendMessage, InvokeLambda (post-contact-summary), DisconnectParticipant

7.2 Error Handling in Flows

Every flow module implements the same error handling pattern: try execution → catch with auto-remediation → retry (max 2) → if still failing, offer graceful agent transfer with error context in screen pop.

8. Amazon Nova Sonic Speech Pipeline

Nova Sonic provides real-time speech-to-speech AI processing. Configuration details:

Parameter	Value
Model	amazon.nova-sonic-v1:0
Streaming Protocol	Bidirectional WebSocket via Connect media streaming
Barge-In	Enabled — citizen can interrupt AI response at any time
End-of-Speech Silence	1200ms (configurable per bureau — IRS uses 1500ms for elderly callers)
TTS Voices	Matthew-Neural (IRS male), Lupe-Neural (IRS bilingual), Joanna-Neural (Treasury), Ruth-Neural (Mint)
Language	en-US primary, es-US secondary (IRS requirement)
Audio Format	PCM 16-bit, 8kHz (telephony) / 16kHz (WebRTC)
Max Turn Duration	30 seconds per citizen turn, 45 seconds per AI response
Session Timeout	30 minutes of inactivity
Fallback	If Nova Sonic unavailable, fall back to Lex Bot + Polly TTS

9. Amazon Bedrock AI Agent Configuration

Each bureau has a dedicated Bedrock Agent. Configuration per agent:

9.1 IRS Agent (Primary — highest volume)

Setting	Value
Agent Name	treasury-irs-agent-v4
Foundation Model	anthropic.claude-sonnet-4-5-v1 (FedRAMP Moderate authorized)
Knowledge Base	kb-treasury-irs-docs-v3 (7 documents: IRS-PUB17, IRS-PUB505, IRS-NOTICE, IRS-INSTALLMENT, IRS-FTA, IRS-FORM, IRS-FILING)
Action Groups	IRS_AccountTools (7 Lambda functions), IRS_FormTools (3 Lambda functions)
Guardrail	treasury-irs-guardrail-v3 (PII: SSN+DOB+ITIN redact, topics: deny investment/political, grounding: 0.70)
System Prompt	You are an IRS virtual assistant for the U.S. Treasury. Verify identity before disclosing any account information. All dollar amounts, dates, and eligibility statements must be Automated Reasoning verified before delivery. Never provide investment advice or political opinions. Always offer live agent transfer if citizen is frustrated.
Session TTL	1800 seconds (30 minutes)
Max Turns	20 conversation turns before mandatory agent offer
Temperature	0.0 (deterministic — critical for factual tax guidance)

10. Knowledge Base and RAG Pipeline

10.1 Document Inventory

Document ID	Title	Bureau	Tokens
IRS-PUB17	Your Federal Income Tax — Filing Requirements, Deductions, Credits	IRS	~48,000
IRS-PUB505	Tax Withholding and Estimated Tax — W-4 Guidance	IRS	~32,000
IRS-NOTICE	Understanding IRS Notices — CP2000, CP05A, LT Series	IRS	~18,000
IRS-INSTALLMENT	Payment Plan and Installment Agreement Guide	IRS	~12,000
IRS-FTA	First-Time Penalty Abatement Policy and Criteria	IRS	~8,000
TD-BONDS	Savings Bond Value Tables and Redemption Rules	TreasuryDirect	~22,000
TD-ACCOUNTS	TreasuryDirect Account Management Guide	TreasuryDirect	~14,000
TOP-OFFSETS	Treasury Offset Program — Debt Collection and Dispute Process	TOP	~16,000
TOP-HARDSHIP	Financial Hardship Exemption Criteria and Application	TOP	~9,000
MINT-PRODUCTS	U.S. Mint Product Catalog and Bullion Pricing	U.S. Mint	~11,000
DE-CARDS	Direct Express Debit Card Program — Benefits and Disputes	Direct Express	~8,000
TREASURY-GENERAL	U.S. Treasury Services Overview and Contact Information	All	~6,000

10.2 RAG Pipeline Steps

Step 1: Citizen utterance → Titan Embed Text v2 → 1024-dimension vector. Step 2: OpenSearch Serverless k-NN search → top 10 chunks. Step 3: Cohere Rerank v3.5 re-scores by query relevance → top 4 retained. Step 4: Contextual Grounding verifies AI response faithfulness to retrieved chunks (threshold 0.70). Step 5: Inject top chunks into Bedrock Agent context as RAG augmentation.

11. Three-Layer Governance System

11.1 Layer 1 — AgentCore Policy (Cedar)

Cedar policies are evaluated BEFORE every tool call and action. Six policies enforce bureau boundaries, taxpayer isolation, refund thresholds, self-service read-only, PII logging prevention, and API allowlisting. Evaluation latency: <10ms.

11.2 Layer 2 — Automated Reasoning

AR provides MATHEMATICAL PROOF that AI responses are factually correct. Six policy domains cover IRS tax guidance (deduction amounts, filing thresholds, credit eligibility), notice explanations (deadline calculations, penalty amounts), payment plans (installment calculation, interest rates), bond rules (maturity dates, redemption values), offset rules (minimum offset amounts, exemption criteria), and identity verification (SSN format, DOB validation). When AR returns INVALID, the response is blocked, the AI is told what was wrong, it regenerates, and AR re-verifies.

11.3 Layer 3 — Bedrock Guardrails

Guardrails provide defense-in-depth: PII Filters (detect and redact SSN, DOB, account numbers, ITIN in both input and output), Topic Filters (deny responses about investment advice, political opinions, other agencies' programs), Content Filters (block harmful, inappropriate, or off-topic content), Contextual Grounding (verify response text is grounded in retrieved KB documents, threshold 0.70), Word Filters (block specific terms that could indicate prompt injection attempts).

12. Deterministic Prediction Engine

The prediction engine is a pure function — no ML model, no training, no drift. Given the same 10 feature inputs, it always produces the same output. This is critical for auditability and regulatory compliance.

12.1 Lambda Implementation

```
# lambdas/prediction/compute-prediction.py
def compute_prediction(features: dict) -> dict:
    weights = {
        'intentComplexity': 0.22, 'sentimentRisk': 0.16,
        'authFriction': 0.08, 'channelComplexity': 0.05,
        'toolHealth': 0.08, 'kbRelevance': 0.08,
        'arConfidence': 0.06, 'repeatCaller': 0.05,
        'queuePressure': 0.12, 'agentAvailability': 0.10
    }
    escalation_risk = sum(features[k] * w for k, w in weights.items())
    escalation_risk = max(0.0, min(1.0, escalation_risk))
    self_service_prob = 1.0 - escalation_risk
    if self_service_prob > 0.62: prediction = 'SELF_SERVICE'
    elif self_service_prob > 0.38: prediction = 'AT_RISK'
    else: prediction = 'ESCALATE'
    return { 'prediction': prediction, 'selfServiceProb': self_service_prob,
             'escalationRisk': escalation_risk, 'features': features }
```

13. Skills-Based Routing Engine

13.1 Intent-to-Skill Mapping

Every intent maps to exactly one skill. The routing Lambda looks up the mapping, evaluates proficiency requirements based on intent complexity and sentiment, checks real-time queue metrics, and decides: route directly, offer callback, apply priority boost, or flag as orphaned.

13.2 Callback Decision Logic

```
# Offer callback when:
# - Queue depth > 8 AND estimated wait > 120 seconds
# - OR available agents at required proficiency == 0
# - AND contact is not VERY_NEGATIVE sentiment (those get priority instead)

# Priority boost applied when:
# - Sentiment is VERY_NEGATIVE
# - OR intent complexity > 0.8 AND auth friction > 0.15
# - Boosted contacts go to queue position 1
```

14. Post-Escalation Agent Lifecycle

When a contact escalates, the system manages a 5-state lifecycle with full observability:

TRANSFERRING: Contact queued for matched agent. Screen pop prepared with AI transcript, KB recommendations, AR results, case history. Queue wait timer starts. CloudWatch log: AgentTransfer.

CONNECTED: Agent accepts, greets citizen with AI context acknowledgment. Q in Connect activates Agent Assist. Contact Lens starts real-time monitoring. CloudWatch log: AgentConnected.

HANDLING: Agent investigates using tools, KB articles, and personal expertise. Agent Assist pushes recommendations as conversation evolves. Contact Lens tracks empathy, talk-over, speaking pace, silence. CloudWatch logs: AgentHandling, AgentAssistPush.

WRAPPING: Call ends, agent enters ACW. Disposition code set (RESOLVED/TRANSFERRED/FOLLOW_UP). Case notes saved. Follow-up scheduled if needed. CloudWatch log: AgentWrapUp.

COMPLETE: ACW finished. Final metrics computed (handle time, FCR, CSAT). Analytics finalized and exported to Data Lake. CloudWatch log: AgentComplete.

15. Real-Time Analytics Pipeline

15.1 Kinesis Stream Architecture

Three Kinesis Data Streams handle different data types: contact-events (contact lifecycle, phase transitions, predictions), contact-lens-output (sentiment scores, entity detection, talk metrics), and agent-events (agent state transitions, assist pushes, quality metrics). Each stream feeds a Kinesis Firehose for S3 Data Lake delivery (Parquet format, partitioned by date/bureau).

15.2 CSAT Prediction Model

Predicted CSAT (1-5 scale) is computed from: base score (SELF_SERVICE: 4.0, AT_RISK: 3.0, ESCALATE: 2.0) + sentiment adjustment (-0.3 to +0.3) + error impact (-0.4 if errors present) + AR verification bonus (+0.2 if all VALID) + channel friction (-0.1 for DTMF auth) + governance pass bonus (+0.1 if all layers passed). Result clamped to 1.0-5.0 range.

16. Flow Operations Center Dashboard

The Flow Operations Center is a React application served via CloudFront, authenticated via Cognito, with real-time data via API Gateway WebSocket.

16.1 Dashboard Components

Component	Description
Header	System status indicator, speed controls, Start/Stop simulation, Guided Demo button, timestamp
Demo Mode Banner	Amber strip showing guided walkthrough instructions when demo mode active
Metrics Bar	8 tiles: IN AI, WITH AGENT, CONTAINED, AT RISK, ESCALATED, CONTAINMENT %, PROCESSED, ERRORS
Contact List (left panel)	Live contacts with phase/agent state indicators, prediction badges, progress bars, 5 visual treatments (inAI, withAgent, callback, contained, agentDone)
CloudWatch Logs Tab	Filterable log viewer with 30+ event types, color-coded actions, status keyword highlighting
Live/Post-Call Analytics Tab	Sentiment timeline, CSAT prediction, categories, outcome card, issues, actions, Agent Assist recommendations, AI insights, post-call summary
AI Agent Tab	Agent reasoning, intent classification, multi-turn conversation state
RAG + Rerank Tab	Query embedding, retrieved documents with rerank scores, grounding verification
Governance Tab	Three-layer pass/fail status, Cedar policy evaluation, AR verification result, Guardrails scan
Routing Tab	Queue metrics dashboard, skill matching, proficiency evaluation, callback decision, priority boost
Sequence Diagram Tab	13-actor technical sequence diagram with phase dividers, demo-mode progressive reveal, color-coded arrows
Errors Tab	Error list with severity, auto-remediation status, error type breakdown
Right Panel	Contact detail, prediction gauge with 10 features, live agent status card, metadata, transcript launch button
Transcript Modal	Full-screen 3-view (Dialog/Sequence/Split) with annotated text (12 entity highlights), agent transcript section
Guided Demo Overlay	Bottom narration panel with thought bubbles, What/Why/Look explanations, step-by-step progression, auto-tab switching

16.2 Real-Time WebSocket API

API Gateway WebSocket endpoint provides real-time data to the dashboard: contact.created, contact.phaseAdvanced, contact.resolved, agent.stateChanged, metrics.updated, log.emitted. Lambda integration functions fan out EventBridge events to connected WebSocket clients.

17. Authentication and Identity

17.1 Citizen Authentication (Contact Flow)

Method	How It Works	Assurance Level
Conversational KBA	Nova Sonic asks citizen to speak last 4 SSN, DOB, ZIP. Lambda verifies against Customer Profiles + IRS Master File.	Medium — good for voice, natural experience
DTMF KBA	IVR prompts citizen to enter SSN last 4, DOB via phone keypad. Same Lambda verification.	Medium — works for citizens uncomfortable with voice
Login.gov SAML	Citizen redirected to Login.gov for SSO. SAML assertion returned to Connect via Cognito federation.	High — government-issued identity, MFA required
ANI Match	Inbound phone number matched against Customer Profiles known numbers.	Low — convenience only, not sole auth for sensitive data

17.2 Dashboard Authentication

Cognito User Pool with SAML federation (Treasury SSO). Role-based access: Ops Admin (full dashboard + controls), Analyst (read-only analytics), Supervisor (real-time monitoring + agent management). Session duration: 8 hours. MFA required for all roles.

18. Security, Compliance, and FedRAMP

18.1 FedRAMP Moderate Compliance Matrix

FedRAMP Moderate requires NIST SP 800-53 Rev 5 controls. Key control families and AWS implementation:

Control Family	Implementation	AWS Services
AC (Access Control)	IAM least-privilege, Cognito MFA, Cedar policies, VPC security groups	IAM, Cognito, AgentCore, VPC
AU (Audit)	CloudTrail all API calls, CloudWatch all app logs, 7-year retention, tamper-evident	CloudTrail, CloudWatch, S3 (log bucket)
CM (Configuration Mgmt)	CDK IaC, Config rules, no manual changes, drift detection	CDK, AWS Config, CloudFormation
IA (Identification/Auth)	Login.gov SAML, KBA verification, ANI matching, session management	Cognito, Lambda, Customer Profiles
SC (System/Comms)	TLS 1.2+ everywhere, KMS encryption at rest, VPC PrivateLink, WAF	KMS, ACM, VPC Endpoints, WAF
SI (System/Info Integrity)	Guardrails (content safety), AR (factual accuracy), Contact Lens (PII redaction)	Guardrails, AR, Contact Lens

18.2 Data Classification

Classification	Examples	Protection
PII — High	SSN, ITIN, DOB, bank account numbers	KMS CMK, Contact Lens auto-redact, Cedar PII logging prevention, Guardrails PII filter
PII — Medium	Name, address, phone number, email	KMS CMK, access logging, role-based access
Tax Return Info (IRC 6103)	Refund amounts, filing status, income data	Bureau boundary Cedar policy, taxpayer isolation, audit logging
Operational	Contact metadata, routing decisions, prediction scores	Encryption at rest, CloudWatch with retention policy

19. Testing and Validation

19.1 Test Strategy

Test Type	Scope	Tools
Unit Tests	Lambda functions, prediction engine, routing logic, Cedar policy evaluation	pytest, jest, Cedar CLI validate
Integration Tests	Bedrock Agent + KB + Rerank, Governance pipeline end-to-end, Connect flow execution	AWS SDK test harness, Bedrock test prompts, Step Functions test execution
E2E Tests	Full contact lifecycle: voice in → AI → governance → prediction → routing → agent → resolve	Connect test framework, Selenium for dashboard, custom contact simulator
Load Tests	1,000 concurrent contacts per bureau, 1 hour sustained	Locust, Artillery, Connect load generator
Security Tests	Prompt injection, privilege escalation, PII leakage, SQL injection, XSS	Custom prompt injection test suite, OWASP ZAP, AWS Inspector
Compliance Tests	FedRAMP control validation, audit log completeness, encryption verification	AWS Audit Manager, custom compliance scripts

19.2 Key Test Scenarios

Scenario 1: IRS refund status voice call — SELF_SERVICE path, AR validates dollar amount, high CSAT.

Scenario 2: IRS penalty abatement with negative sentiment — ESCALATE path, P3 agent assigned, warm handoff with screen pop.

Scenario 3: TOP hardship exemption with AR INVALID — governance catches incorrect exemption threshold, corrects, re-verifies, delivers correct answer.

Scenario 4: Cross-bureau tool call attempt — Cedar policy DENIES IRS agent calling TreasuryDirect API, logs violation.

Scenario 5: Queue pressure callback — high queue depth triggers callback offer, citizen accepts, callback made within estimated time.

Scenario 6: Nova Sonic fallback — simulate Nova Sonic unavailability, verify graceful fallback to Lex + Polly.

20. Appendix: Complete File Manifest

Every file that Kiro CLI should generate, organized by directory:

20.1 Infrastructure (CDK)

```
infrastructure/lib/stacks/network-stack.ts
infrastructure/lib/stacks/data-stack.ts
infrastructure/lib/stacks/auth-stack.ts
infrastructure/lib/stacks/connect-stack.ts
infrastructure/lib/stacks/bedrock-stack.ts
infrastructure/lib/stacks/agentcore-stack.ts
infrastructure/lib/stacks/governance-stack.ts
infrastructure/lib/stacks/nova-sonic-stack.ts
infrastructure/lib/stacks/analytics-stack.ts
infrastructure/lib/stacks/monitoring-stack.ts
infrastructure/lib/stacks/dashboard-stack.ts
```

20.2 Lambda Functions

```
lambdas/auth/verify-identity.py
lambdas/auth/login-gov-callback.py
lambdas/auth/ani-match.py
lambdas/tools/irs/get-refund-status.py
lambdas/tools/irs/get-notice-details.py
lambdas/tools/irs/calculate-payment-plan.py
lambdas/tools/irs/check-penalty-eligibility.py
lambdas/tools/irs/get-filing-deadlines.py
lambdas/tools/irs/request-w2.py
lambdas/tools/irs/get-amended-return-status.py
lambdas/tools/td/get-bond-value.py
lambdas/tools/td/get-account-access.py
lambdas/tools/td/get-maturity-date.py
lambdas/tools/td/process-bond-purchase.py
lambdas/tools/top/get-offset-inquiry.py
lambdas/tools/top/initiate-dispute.py
lambdas/tools/top/check-hardship-exemption.py
lambdas/tools/top/trace-payment.py
lambdas/tools/mint/get-order-status.py
lambdas/tools/mint/get-product-availability.py
```

```
lambdas/tools/mint/get-bullion-pricing.py
lambdas/tools/de/get-card-balance.py
lambdas/tools/de/process-dispute.py
lambdas/tools/de/request-replacement-card.py
lambdas/prediction/compute-prediction.py
lambdas/routing/route-contact.py
lambdas/routing/queue-metrics.py
lambdas/governance/governance-pipeline.py
lambdas/governance/ar-validate.py
lambdas/analytics/realtime-analytics.py
lambdas/analytics/post-call.py
lambdas/agent/lifecycle-tracker.py
```

20.3 Policy Files

```
cedar-policies/taxpayer-isolation.cedar
cedar-policies/bureau-boundaries.cedar
cedar-policies/refund-threshold.cedar
cedar-policies/self-service-readonly.cedar
cedar-policies/pii-logging-prevention.cedar
cedar-policies/api-allowlist.cedar
ar-policies/irs-tax-guidance.json
ar-policies/irs-notice-explanations.json
ar-policies/irs-payment-plans.json
ar-policies/td-bond-rules.json
ar-policies/top-offset-rules.json
ar-policies/identity-verification.json
guardrails/irs-guardrail.json
guardrails/td-guardrail.json
guardrails/top-guardrail.json
guardrails/mint-guardrail.json
guardrails/de-guardrail.json
```

20.4 Connect Flows

```
connect-flows/irs-main.json
connect-flows/td-main.json
connect-flows/top-main.json
connect-flows/mint-main.json
connect-flows/de-main.json
```

```
connect-flows/modules/init-module.json
connect-flows/modules/auth-module.json
connect-flows/modules/ai-agent-module.json
connect-flows/modules/govern-module.json
connect-flows/modules/route-module.json
connect-flows/modules/resolve-module.json
```

20.5 Knowledge Base Documents

```
knowledge-base/IRS-PUB17.pdf
knowledge-base/IRS-PUB505.pdf
knowledge-base/IRS-NOTICE.pdf
knowledge-base/IRS-INSTALLMENT.pdf
knowledge-base/IRS-FTA.pdf
knowledge-base/TD-BONDS.pdf
knowledge-base/TD-ACCOUNTS.pdf
knowledge-base/TOP-OFFSETS.pdf
knowledge-base/TOP-HARDSHIP.pdf
knowledge-base/MINT-PRODUCTS.pdf
knowledge-base/DE-CARDS.pdf
knowledge-base/TREASURY-GENERAL.pdf
```

20.6 Dashboard

```
dashboard/src/App.tsx           # Main Flow Ops Center component
dashboard/src/hooks/useWebSocket.ts  # Real-time data connection
dashboard/src/components/         # Tab components, contact list, metrics
dashboard/public/                 # Static assets
dashboard/package.json
dashboard/tsconfig.json
```

— End of Kiro CLI Implementation Specification —